ATELIER PRATIQUE

— Préparer l'environnement Laravel

- ✔ **Installer PHP**
- ✔ **Installer Composer**
- ✔ **Installer Laravel**
- ✔ **Créer un projet Laravel**
- ✔ **Comprendre l'arborescence Laravel**
- ✔ **Utiliser Artisan**
- ✔ **Lancer un serveur Laravel**

ÉTAPE 1 — Vérification / Installation de l'environnement

Objectif

S'assurer que **PHP** et **Composer** sont installés et fonctionnels.

Vérifier PHP

Action

Ouvrez le **terminal (CMD)** et tapez :

php --version

✔ **Résultat attendu**

Une version s'affiche (ex : PHP 8.x)

X **Si PHP n'est PAS installé**

Téléchargez PHP ici (lien officiel) :
https://windows.php.net/download/

➤ **Conseils d'installation PHP (Windows)**

- Télécharger **ZIP**

- Décompresser dans : C:\PHP
- Ajouter C:\PHP dans **Variables d'environnement → PATH**
- Vérifier avec :

php --version

À retenir

Laravel est écrit en PHP → PHP est obligatoire

Vérifier Composer

➤ **Action**

Dans le terminal :

composer --version

✔ **Résultat attendu**

Une version s'affiche (ex : Composer 2.x)

X Si Composer n'est PAS installé

Téléchargement officiel :
https://getcomposer.org/download/

➤ **Pendant l'installation**

✔ Vérifier que Composer pointe vers :

C:\PHP\php.exe

À retenir

Composer est le gestionnaire de dépendances de Laravel

ÉTAPE 2 — Création d'un projet Laravel

Objectif

Créer un **nouveau projet Laravel fonctionnel**.

Action

- ✓ Ouvrez le terminal
  Placez-vous dans le dossier serveur web :

cd C:\xampp\htdocs

ou

cd C:\wamp64\www

- ✓ Créez le projet :

composer create-project laravel/laravel projet_test

Attendez la fin du téléchargement (important)

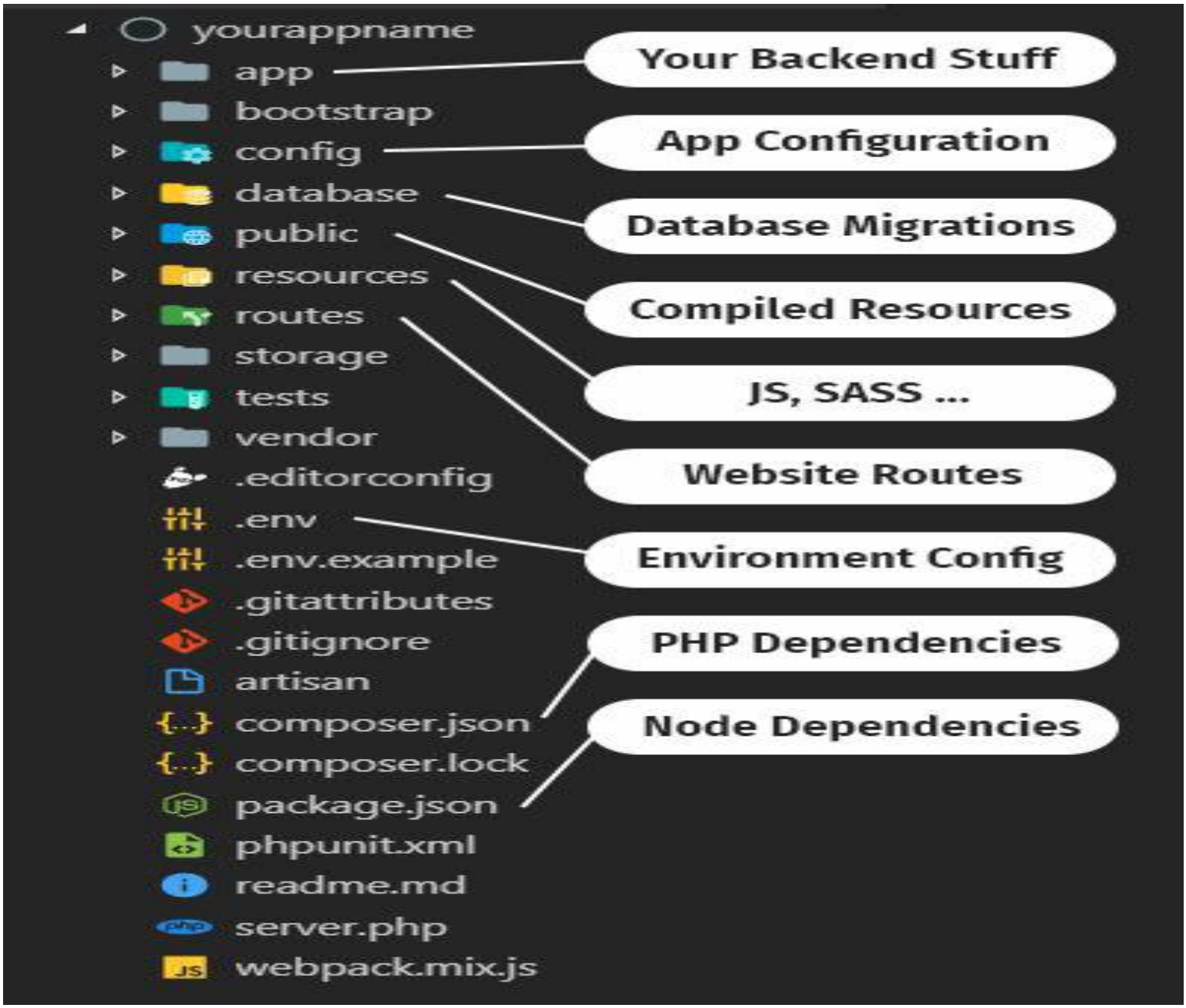
À retenir

Cette commande télécharge Laravel + toutes ses dépendances

```
➜  code composer create-project laravel/laravel test dev-develop
Installing laravel/laravel (dev-develop d6acad21cb2288713d9c09a31f9b4ab86f116039)
  - Installing laravel/laravel (dev-develop develop): Cloning develop from cache
Created project in test
> @php -r "file_exists('.env') || copy('.env.example', '.env');"
Loading composer repositories with package information
Updating dependencies (including require-dev)
Package operations: 71 installs, 0 updates, 0 removals
  - Installing vlucas/phpdotenv (v2.5.1): Loading from cache
  - Installing symfony/css-selector (v4.1.3): Loading from cache
  - Installing tijsverkoyen/css-to-inline-styles (2.2.1): Loading from cache
  - Installing symfony/polyfill-php72 (v1.9.0): Loading from cache
  - Installing symfony/polyfill-mbstring (v1.9.0): Loading from cache
  - Installing symfony/var-dumper (v4.1.3): Loading from cache
  - Installing symfony/routing (v4.1.3): Loading from cache
  - Installing symfony/process (v4.1.3): Loading from cache
  - Installing symfony/polyfill-ctype (v1.9.0): Loading from cache
  - Installing symfony/http-foundation (v4.1.3): Loading from cache
  - Installing symfony/event-dispatcher (v4.1.3): Loading from cache
  - Installing psr/log (1.0.2): Loading from cache
  - Installing symfony/debug (v4.1.3): Loading from cache
  - Installing symfony/http-kernel (v4.1.3): Loading from cache
  - Installing paragonie/random_compat (v9.99.99): Loading from cache
```

ÉTAPE 3 — Découverte de l'arborescence Laravel (PARTIE CLÉ)

Ouvrez le dossier projet_test dans Visual Studio Code



📂 /app — Le cerveau de l'application

Rôle

Contient toute la logique métier.

📁 Sous-dossiers importants :

- Http/Controllers → reçoit les requêtes
- Models → communication avec la base de données

Analogie

app = le cerveau du site

/routes — Les URLs

Contient toutes les routes du site

Fichier principal :

- web.php

Exemple :

```
Route::get('/', function () {
   return view('welcome');
});
```

Analogie

routes = GPS du site

📂 **/resources — L'affichage**

Contient :

- views → pages HTML (Blade)
- CSS / JS

Analogie

resources = vitrine du site

📂 /public — Point d'entrée

Contient :

- images
- CSS / JS
- index.php

IMPORTANT

Le navigateur ne voit QUE ce dossier

📂 /config — Réglages

Contient :

- base de données
- mail
- cache
- sessions

Analogie

config = tableau de bord

📂 /storage — Stockage interne

Contient :

- logs
- cache
- fichiers uploadés

Commande importante :

php artisan storage:link

Analogie

storage = archive interne

📂 /database — Base de données

Contient :

- migrations
- seeders

Analogie

database = plan de construction

📄 composer.json — Carte d'identité

Contient :

- version PHP
- dépendances

Analogie

composer.json = CV du projet

ÉTAPE 4 — Laravel Artisan (CLI)

Objectif

Utiliser les commandes Laravel.

Actions

```
php artisan list
php artisan --version
php artisan route:list
```

À retenir

Artisan est le tableau de commande de Laravel

ÉTAPE 5 — Lancer le projet Laravel

Actions

php artisan key:generate

php artisan serve

Navigateur

http://localhost:8000